kid_buddy — 会讲故事的儿童 AI 伙伴

2026 首届 openvela AI 硬件开发者大赛 · 技术报告

1、信息表

项目	内容
作品名称	kid_buddy（会讲故事的儿童 AI 伙伴）
队伍名称	地铁口野狗联盟
团队分工	5 人，分工见下表
选题方向	AI 硬件产品创新

成员	技术方向	核心职责
H1：丁培钊	音频与 BSP 驱动	录音与播放链路（ALSA 采集 / 播放）、codec 二次打开无声的驱动修复、单麦克风增益与 VAD 阈值调参
H2：邢铭航	固件与系统集成	固件编译与版本管理、WiFi / 外设配置、系统集成、界面帧预算与性能调优
S1：秦朝	云端服务与后端	MQTT Broker 与通道联调、家长 Web 控制台（单文件 H5）
S2：李浩翔	端侧 UI/UX	LVGL 整体 UI、多套角色形象设计、动画系统、触摸交互
S3：黎煌华	AI 引擎与端侧逻辑	System Prompt 与四套角色人设设计、`story-rpg` 技能、故事模式与剧情会话逻辑

2、摘要

kid_buddy 是一台给 6–12 岁孩子的语音 AI 玩伴，跑在 openvela + ai_agent 上，硬件是一块 巴掌大的 Gemini-S1，套在 3D 打印的「翻开的书页」外壳里。

它针对的痛点很具体：儿童故事机内容出厂即固定、只会单向播放；通用语音助手则是「你问它 才答」，孩子得先喊醒它、还得会组织问题，门槛全压在孩子这边。kid_buddy 把「主动」和 「执行」都

交给 ai_agent 框架——提醒是模型自己调 cron 工具建立的定时任务，故事讲一半 孩子走开了 Agent 会主动开口续讲，剧情走分支且与自由问答各存一份会话历史、断电再开机 接着上次那条线讲。

交互上做了一个反常识的选择：屏幕上一个字都没有，整块屏是一张用圆和弧线画的表情脸， 把二十来个设备状态翻译成六种表情；底部三分之一的字幕板是给旁边大人看的旁证，不是给 孩子的提示——语音一旦哑掉，那张难过的脸是唯一能证明它还活着的东西。

代码与固件在本队专属仓内，云侧只用大赛提供的 MiMo（LLM / ASR / TTS）。

3、正文

3.1 绪论

3.1.1 项目背景与问题定义

应用场景。 6–12 岁的孩子在家里的非学习时段（放学后、睡前）需要有内容的陪伴。 这个年龄段的孩子识字量有限、不会打字、说不出「请重复上一句」这种指令式表达；屏幕 也不是给他们的——他们不读，他们看。

痛点一：现有的儿童设备都是单向的。 故事机按下按钮就播放，内容出厂即固定，孩子 没有选择权，想让故事里的主角往回走是做不到的。而这类设备真正被孩子喜欢的部分恰恰是 能参与——所以孩子会反复听同一集，因为那是唯一能预期的东西。

痛点二：通用语音助手是「被动问答」的。 你不问，它就永远静默。孩子对一台设备开口 是有门槛的：要先意识到它能听懂、要组织好语言、还要先喊对唤醒词。门槛叠加的结果是， 孩子玩两次就不玩了。

痛点三：云端语音链路一旦失败，廉价设备会「假装无事发生」。 网络抖动、服务端握手 失败、音频通道被抢——这些对大人来说是「稍等重试」，对孩子来说是「玩具突然不理我了」， 而两者在行为上完全一样：你说完话，它没有任何反应。

3.1.2 技术难点

1. **单麦克风板子上，唤醒、识别、结果匹配全要在应用层自己串。** 这块板没有本地 KWS 引擎，唤醒只能在 ASR 转写结果上做匹配；而转写是中英混说，空格、标点、大小写都不 固定。
2. **采集和播放共享同一条 codec DMA 通道。** 麦克风开着的时候去开扬声器，会直接报 `request dma chan failed`，提醒和回复都会变成哑的。
3. **屏幕是整屏单缓冲的（FULL 渲染模式）。** 任何一处失效，LVGL 都要重绘并刷整块屏 —— 320×240×2 = 153,600 字节，而 SPI1 同时还被音频 DMA 抢。所谓「只重绘脸那块」的 常规优化在这块板上一分钱都省不下来。

4. **板子没有 RTC**。 系统时钟停在 1970，HTTPS 证书的 notBefore 校验对每一张服务端 证书都失败，ASR / LLM / TTS 整条链路会全哑，而日志里只有 TLS 错误。
5. **让模型真的去「执行」，而不是把执行写在应用代码里**。 提醒不能是我们写死的正则 解析，故事也不能是我们写死的状态机——否则这就只是又套了个大模型的语音助手。

3.1.3 创新点

交互层：把状态翻译成表情，而不是翻译成文字。 孩子不读字，所以整块屏是一张脸： 六张基础表情 × 六种头顶道具，覆盖二十来个设备状态。底部三分之一保留三行字幕板，但它 的定义是「给大人的旁证」——语音一旦失败，屏幕上的难过表情 + 一句中文安抚语是唯一的 告知。这个取舍是有代价的（没有文字兜底，TTS 哑了这轮回复就丢了），所以失败路径必须 显式地演出来，不能靠中性表情含糊过去。

内容层：把「怎么讲一个分支故事」写成 Agent 能读的 Skill，而不是应用里的状态机。 `story-rpg` 技能是一份放在 `/data/agent/skills/` 的 Markdown，写的是「怎么起头、每轮 怎么给选项、选项要用完整口语不能念 ABC、怎么续、怎么收尾」。换讲法改 Markdown 即可， 不用重新编译固件。

机制层：剧情与闲聊的会话隔离。 孩子中途问一句「恐龙为什么会灭绝」， 不会把剧情带 跑偏——两个会话在系统里各存一份历史。这是「能一直玩下去」和「玩两轮就乱」的分界线。

工程层：把云端链路的瞬时故障挡在孩子看不见的地方。 TLS 逐 IP 回退、指数退避重试、 chunked 响应读完即停、回复卡死的看门狗自恢复、开机自动校时。这些不是锦上添花——它们 决定了这个玩具是「偶尔不灵」还是「坏了」。

3.2 系统方案设计

3.2.1 系统总体架构



数据流自下而上：孩子说话 → VAD 掐句 → MiMo ASR 转写 → 唤醒匹配或追问窗口放行 → `velaclaw_ask()` 进 Agent 主循环（带工具表）→ 模型决定直接回答、调 cron 工具、还是读 `story-rpg` 技能 → 回复流式回吐 → 按句预取 MiMo TTS → ALSA 播放，同时脸谱切换到说话表情。

3.2.2 方案论证与选型

开发板选型理由。 大赛提供的 Gemini-S1 是唯一同时具备麦克风、扬声器、LCD 和触屏的 板子，而这四样恰好是这个作品的最小硬件集：单麦决定了唤醒方案必须走「ASR 转写后匹配」， LCD + 触屏决定了交互可以用一张脸 + 四条书签带完成。它同时也是难度来源——没有本地 KWS 引擎、没有 RTC、采集与播放共享 DMA，这三条都在 3.1.2 里列过。

端 / 云职责划分。 这是本作品最需要说清楚的一处：

能力	在哪一侧	理由
唤醒判定	**端侧**	本地能量阈值 VAD + 归一化字符串匹配，不需要联网，省 Token，也避免「说一句话就把音频传上云」
语音识别 / 合成 / 对话	云端（MiMo）	端侧算力与内存都放不下可用的模型

意图判断与动作决策	云端模型 + **端侧工具执行**	模型决定「要做什么」，动作（建定时任务、写文件）在板子上真实发生
会话历史 / 剧情进度 / 提醒任务	**端侧落盘**	断网也不能丢；重启后要接着上次的剧情讲
表情与状态机	**端侧**	实时性要求（100 ms 级），且断网时更要工作

断网 / 弱网降级策略。

- **断网（板子知道自己没网）：脸显「困倦」而不是「难过」，头顶冒泡泡**。 这是刻意 区分的：「帮不上忙」和「我做错了事」对孩子不是一个意思，一张难过的脸会让孩子以为 是自己把玩具弄坏了。不做无意义的静默重试；网络恢复后由 `network_is_connected()` 轮询自动接回。
- **联网但这一轮失败（服务端某个后端 IP 握手坏窗口、5xx、请求根本没发出去）：脸显 「难过」并播一句写死的中文安抚语**。 这一条优先级高于上一条——真的失败了就要认， 在这个没有文字的界面上，这是唯一还能告诉孩子「它坏了，不是它不想理你」的方式。
- 弱网瞬时故障（服务端某个后端 IP 握手必失败）：TLS 连接对域名解析出的**每个 IP 逐次 尝试**，而不是只试第一个就放弃。
- 服务端返回 5xx / 连接被拒：TTS 请求最多重试 3 次、间隔 1.5 s 起；握手成功但响应 完整性校验失败，则整个请求重试而不是接受半截数据。
- 响应流读不完：对 keep-alive 的 chunked 响应，读到 body 完整即主动停止，不等服务端 关连接。
- 开机兜底：开机约 4 秒后先播一句**写死的**欢迎语。它不走 LLM，所以云端首字再慢孩子 也不会干等；联网之后才发起第二轮「查会话历史、接着上次那条线讲」的探测，没网就整个 跳过（那一轮的失败只会换来一句道歉，反而把上一次的故事顶掉）。

与备选方案的对比。

备选	为什么不选
纯文字聊天界面（卡片 + 按钮 + 聊天气泡）	目标用户不识字。这是本作品第一版的样子，实测孩子在屏幕上找不到任何可读的东西
端侧小模型做唤醒（KWS）	板子没有可用的本地 KWS，且引入模型会让内存和算力预算失控；改用「ASR 转写后匹配」把成本压到几乎为零
提示词里写死角色人设，不做 Skill	换讲法要重新编译固件；而且那样「讲故事」的能力就没有留在框架里，和本赛题「用 Skill 扩展 Agent」的要求背道而驰
剧情进度只放在对话历史里	中途孩子问一句别的，剧情状态就被闲聊污染了

3.2.3 关键模块设计

（1）语音链路。 采集用本地能量阈值 VAD 掐出一句话（20 ms 一帧，起止阈值带迟滞），只在没有回复在播的时候才开麦克风。识别送 MiMo ASR。合成用 MiMo TTS，**一次回复只开关一次音频设备**，按句流水线预取——转换下一句和播放当前句并行，首字延迟约减半。整条链路上「谁是当前这句话的接收方」由一个忙标志门控，避免采集与播放同时占用 codec DMA。

（2）唤醒。 板子没有本地 KWS，唤醒是在 ASR 转写结果上做归一化匹配：大小写折叠、空格与标点当分隔符、并同时认几个常见音译写法，所以「你好，openvela」「你好 openvela」「Hello，OpenVela」都能唤醒。唤醒词按大赛规定使用「你好，openvela / Hello, openvela」。

（3）追问窗口。 回复结束后的 12 秒内，孩子直接说「第二个」「继续」就能接话，不用再喊唤醒词。小孩说不出「请重复上一句」，这个窗口是替他们省事的。窗口内收到的音频不再走唤醒判定，直接进对话。

（4）剧情会话管理。 进入故事模式时，这一路的请求带一个独立的会话标识，ai_agent 的会话管理器把它的历史单独存一份。进出故事模式由语音触发（「我们来玩冒险游戏」/「不玩了」），状态落盘，中途断电再开机还能接着上次的剧情。

（5）脸谱状态机。 六张基础表情（平静 / 高兴 / 惊讶 / 专注 / 难过 / 困倦）× 六种道具。状态按优先级阶梯判定：ASR 退避 → 出错难过 → 提醒铃铛 → 上报成功 → 主动搭话挥手 → 正在听（音量弧）→ 识别中 → 说话中（开合嘴）→ 思考中 → 没听清 → 无网络（困倦）→ 待机。动画按状态分配帧预算（见 3.5.2），待机时每 tick 一个 LVGL 调用都不发。

（6）各节点的角色与运行的系统。 本作品是一个三节点的方案，但**只有板子在跑 openvela**，另两个节点刻意做成了零安装：

节点	跑在哪	承担什么	为什么这么分
设备端	Gemini-S1 / **openvela**（NuttX）	全部实时动作：脸谱界面、VAD 采集、唤醒匹配、TTS 播放、会话与剧情落盘、定时任务执行	实时性（100 ms 级）和断网可用性都要求它在本地
家长端	手机 / 电脑**浏览器**，一个单文件 H5	看孩子与设备的往来、给孩子设提醒	不装 App、不上应用商店，扫开即用；它只是个 MQTT 客户端
云侧	Xiaomi MiMo	ASR / 对话 / TTS 三个接口	端侧算力与内存放不下可用的模型

家长端和板子之间只隔着 MQTT（topic `agent/in` / `agent/out`）。**普通的收发消息用的就是 ai_agent 原有的入站 / 出站格式**（`{"type":"message","content":…,"chat_id":…}`），设备端一行没改；将来要换成手机原生 App、或者接一个智能音箱，这条路仍然不用动设备端。

（7）孩子分享的冒险：家长端的长期档案。在上面那套「家长端只读状态」之外，这是唯一一条**给家长看内容**的链路——而且只传孩子主动同意分享的部分。

- **分享闸门做在板子上，不在网页上**。故事会话（`kid_buddy:story`）是孩子自己的完整剧情，用于续讲；孩子说「把这次冒险放进故事书 / 分享给妈妈」时，板子按这次冒险的起始时间把对应的会话行切出来，追加进独立的分享档案 `kid_buddy:share`。家长端同步的是后者，所以即使网页想多要，也拿不到没分享的剧情。
- **取数不经过 Agent**。家长端发 `{"type":"history","target":"kid_buddy:share","chat_id":"parent"}`，MQTT 通道直接调 `session_get_history_json_ts()` 读分享档案回发。这件事是纯取数据，绕一次 LLM 既慢又会被模型改写，直连更合适。带 `ts` 是给家长端按天分组和日报用的；LLM 那条路径仍然用不带 `ts` 的版本，避免把未知字段塞进 messages 数组。
- **传输是切片限速的**。MQTT 的发送缓冲只有 `AGENT_MQTT_BUF_SIZE` = 1024 字节，一条历史塞不下；而 `mqtt_publish` 是往缓冲里写，背靠背连发会撞上 `MQTT_ERROR_SEND_BUFFER_IS_FULL`。做法是切成 384 字节的片、每片独立 base64，**由已有的 MQTT sync 线程每 200 ms 发一片** —— 借现成的节奏限速，既不开新线程也不会撑爆缓冲。实测十轮冒险（3621 字节）切成 10 片，单片 payload 最大 607 字节。
- **分片拼接有个必须避开的坑**。各片的 base64 字符串**不能拼接后整体解码**：每片字节数不整除 3 时末尾带 `=` 填充，拼起来解码是错位的（实测 3621 字节只剩 767 字节活下来）。正确做法是每片各自解码成字节、再把字节拼起来，最后整体做一次 UTF-8 解码。这样片与片之间哪怕正好切断一个汉字也不影响——字节流拼回去就复原了。
- **长期档案存在手机那侧**。板子每个会话只保留最近 `AGENT_SESSION_MAX_MSGS`（20）条并滚动截断，所以每次同步是把新出现的合并进家长端本地，按 `(role, ts, content)` 去重。**板子是窗口，手机是档案**——孩子玩得越久，手机上的记录比板子上能留住的越多。

（8）世界角色主动邀请孩子回来。产品上这是「主动技能」：板子不等唤醒词，自己开口。实现上它复用了已有的空闲检测，但有三个不显然的约束：

- **邀请要落在故事会话里**。孩子这时已经退出故事模式了，如果照普通主动搭话发到默认会话，模型看不到上次讲到哪，只能说一句空泛的「回来玩吧」。所以这一轮带一个显式的 `chat_id` 覆盖，直接发到故事会话；孩子一接话，下一轮把故事模式重新打开，剧情从断开的地方继续，分享时也不会把前半截切掉（`g_story_started_at` 保持不变）。
- **开口是有代价的，门槛必须写死**。静默 5 分钟才邀请；两次至少隔 20 分钟、一天最多 3 次；睡前时段、时钟未校准、没连网一律不说话。晚上十点板子突然有人声，比它一直不吭声糟糕得多。

- **接话有保质期**。 邀请后 10 分钟内孩子开口才自动回到故事模式；超过这个窗口， 否则他随口问一句「今天天气怎么样」就会被拽进剧情。

同一次改动顺手修了空闲续讲的老问题：主动那一轮会把空闲时钟归零，而原来的实现没有 「这个离开已经问过」的标志，孩子不理它就会每 60 秒重复问一次。现在一个离开只问一次。

3.3 核心算法与技术原理

3.3.1 AI 算法实现

端侧模型：不部署。 板子的内存与算力预算放不下一个可用的语音或语言模型，所以端侧 不跑神经网络：唤醒是确定性算法（能量 VAD + 字符串归一化匹配），表情是几何绘制。这条 选择换来的是断网可用性和零端侧推理延迟——代价识别与合成必须联网，因此失败路径必须 显式可见（见 3.1.3）。

云端模型：全部使用大赛 Xiaomi MiMo。 三个接口：

用途	模型	接口	鉴权
语音识别	`mimo-v2.5-asr`	HTTPS POST，音频以 `data:audio/wav; base64,` 内联，语言 `zh`	Bearer API Key
对话 / 工具调用	MiMo 对话模型	`https://api.xiaomimimo.com/v1/chat/completions`（OpenAI 兼容）	Bearer API Key
语音合成	`mimo-v2.5-tts`	`https://token-plan-cn.xiaomimimo.com/v1/chat/completions`，默认音色「冰糖」，故事模式按角色切换（冰糖 / 茉莉 / 苏打 / 白桦），输出 `pcm16`	Bearer API Key

API Key 与模型名由 `/data/agent/config/config.json` 配置（可用 NSH 命令在线改）， 不写死在固件里。对话请求走流式（SSE），首句到即开始合成，不等整段生成完。

3.3.2 关键机制设计

（1）字幕排版：行宽预算必须自己算，而且要算对。 LVGL 的标签在两种情况下都会断行： 格子用完了，以及遇到 `\n` 和空格。只按「格子数 × 行数」估算会多算出一行，而多出来的 那一行不报错、不画出来、安静消失——症状是「回复念到一半就没了」。所以排版自己做： 逐字断行并显式写入 `\n`。

配套的是全角判断。实测字体 `lv_font_simsun_16_cjk` 的度量**精确**是 ASCII 8.00 px、 CJK 16.00 px，但常见的「U+2000 以上是全角」是错的——实际有 128 个更小的码点也是全角 （`° ± × ÷`、希腊字母、西里尔字母），把它们算成半角会让整行宽 8 px，恰好够 LVGL 再折 一次行，于是又多出一行被裁掉。

（2）字幕滚动：把「裁切」从限制反过来当工具用。 提示词要求模型答 100 字，而两行 33 格只有 66 格——上限是屏幕给的，不是模型给的。做法是：正文标签允许长到 8 行，前面放 一个正好两行高的普通对象当窗口，标签是它的子对象，每 2.5 秒往上移一行，走到底回到开头。

这条能成立的前提是从源码里读出来的两条性质：LVGL 画子对象时只取**父对象的矩形**做 交集（除非父对象带 `LV_OBJ_FLAG_OVERFLOW_VISIBLE`，我们故意不加——裁切正是这个功能的 实现手段）；而标签自己的换行宽度用的是它自己的坐标，不受父对象裁切影响。另外 `lv_obj_set_y()` 在值没变时会提前返回，所以静止的标签每个 tick 零成本——这一点很重要， 因为 FULL 渲染模式下任何一次写入都是 153 KB 的全屏刷新。

滚动的起算点是「文本最后一次变化」，不是流结束的 final 事件：出错路径根本不发 final， 等它会让半截回复永远卡在开头。

（3）剧情会话隔离。 见 3.2.3（4）。实现上是一次请求带一个会话标识，历史由框架按 标识分别落盘。

（4）设备状态 → 表情的映射。 应用只往界面暴露若干只读的 `volatile` 观察点（是否在 听、音量、是否在识别、是否在说、是否出错、是否连网……），`face_apply()` 按优先级阶梯 把它翻译成表情与道具。这样界面重做不会碰到任何控制流。

3.3.3 openvela 系统能力的深度运用

大赛要求「至少落地图形 / AI / 多媒体三项核心能力之一」，本作品三**项全部落地**：

能力	落点	用到的 openvela 组件
图形	整块屏的 LVGL 脸谱界面、字幕排版与滚动、触摸书签带	LVGL 9.1、NuttX LCD 驱动（`/dev/lcd0`）、触摸屏输入
AI	ai_agent 的 Agent 主循环、工具调用、Skill 体系、会话与记忆	`packages/ai_agent`、MiMo LLM/ASR/TTS
多媒体	单麦录音、TTS 播放、按句流水线、DMA 通道仲裁	NuttX ALSA、AW sun8iw20 audio codec 驱动

底层还用到：mbedTLS（HTTPS/WSS）、lwIP（TCP/IP）、MQTT 客户端（家长端上报）、NAND 文件 系统（配置、会话、剧情与提醒任务落盘）、NSH（调试与在线配置）。

对 openvela 的优化、拓展与改进建议。 开发过程中定位到的问题，能改的都提了 PR：

仓	PR	内容	性质
`packages_ai_agent`	#33	新增 MiMo ASR / TTS 后端；修音频通道与播放链路	能力扩展
`packages_ai_agent`	#34	修：外发队列被阻塞时提醒不再哑掉	缺陷修复
`packages_ai_agent`	#35	多角色扮演、追问窗口、剧情会话隔离、`story-rpg` 技能	能力扩展
`packages_ai_agent`	#36	Agent 单实例、MQTT 儿童确认上报、CLI 守护互斥	能力扩展 + 缺陷修复
`vendor_allwinnertech`	#21	Gemini-S1 板级支持；codec 每次打开无条件重配 DAC 时钟的修复	板级适配 + 驱动缺陷修复

上游评审有 CODEOWNERS 闸门，参赛者无法自行合并，截至交赛这 5 个 PR 都仍在评审队列中。 为使作品**不依赖上游合并即可从源码构建**，`packages/ai_agent` 的 39 个文件与 `vendor/allwinnertech` 的 3 个文件的成品随本仓 `upstream/` 目录交付，manifest 通过 `<copyfile>` 在 `repo sync` 结束后按原路径覆盖回对应工作区。因此 `repo sync` 得到的树 与 `image/` 内固件的源码状态一致。此处使用 `<copyfile>` 而非 `<linkfile>`：后者在目标 位置已存在同名普通文件时不会覆盖（`repo` 的 `removedirs()` 只清理符号链接与空目录， 遇到普通文件即停止），仅打印一行错误日志，工作区中保留的仍是上游原文件。

给 openvela 的改进建议（都来自这块板子上真实踩到的坑）：

6. **构建系统不跟踪头文件依赖**。 改过 `.h` 之后链接的是旧目标文件，症状是结构体字段 读出垃圾值（我们遇到的是「读出 240」）。这类问题定位成本极高，建议在应用层构建规则 里补上 `.h` 依赖。
7. **构建系统只把 `etc/init.d/rcS` 列为依赖，而它的全部内容是 `#include "rcS.nsh"`**。 改 `rcS.nsh` 永远不触发重新生成，但 `make` 报成功——镜像里还是旧脚本。
8. **LCD 驱动的 `#if/#elif` 分支选择过于隐蔽**。 板级配置里同时打开 `CONFIG_LCD_FRAMEBUFFER` 会让 `lcddev_register()` 变成死代码、`/dev/lcd0` 根本不注册，表现为整屏白屏而日志里 没有明确报错。建议该分支在编译期给出一条显式告警。
9. **`lv_nuttx_lcd.c` 里 `GETAREAALIGN` ioctl 失败时静默继续**。 该 ioctl 失败会让 `rounder_cb` 把每个 flush 面积压成 0 宽，也就是永久白屏，而代码只 `perror`。建议失败 时直接报错退出。

10. **无 RTC 的板子建议在系统层提供时间可信标志。** 我们最终是在应用里加 `seed_clock_if_unset()` 并在未校时拒绝设置绝对时间的提醒，但这类问题对所有无 RTC 的 板子都成立，放在系统层解决更合适。

3.4 系统实现

3.4.1 软件 / 固件架构

作品主体是一个文件、一个进程：`app/kid_buddy/kid_buddy.c`（约 4500 行 C），在 NuttX 上 作为一个独立应用由启动脚本拉起。它自己不起 Agent 主循环，而是通过 `velaclaw_ask()` 把 请求推进消息总线，由一个常驻的 ai_agent 实例处理——包括工具调用、会话管理、记忆落盘与 定时任务。两者之间只有两个方向的接口：请求（ask）与**非请求的消息推送**（notify 回调， 提醒就是从这里进来的）。

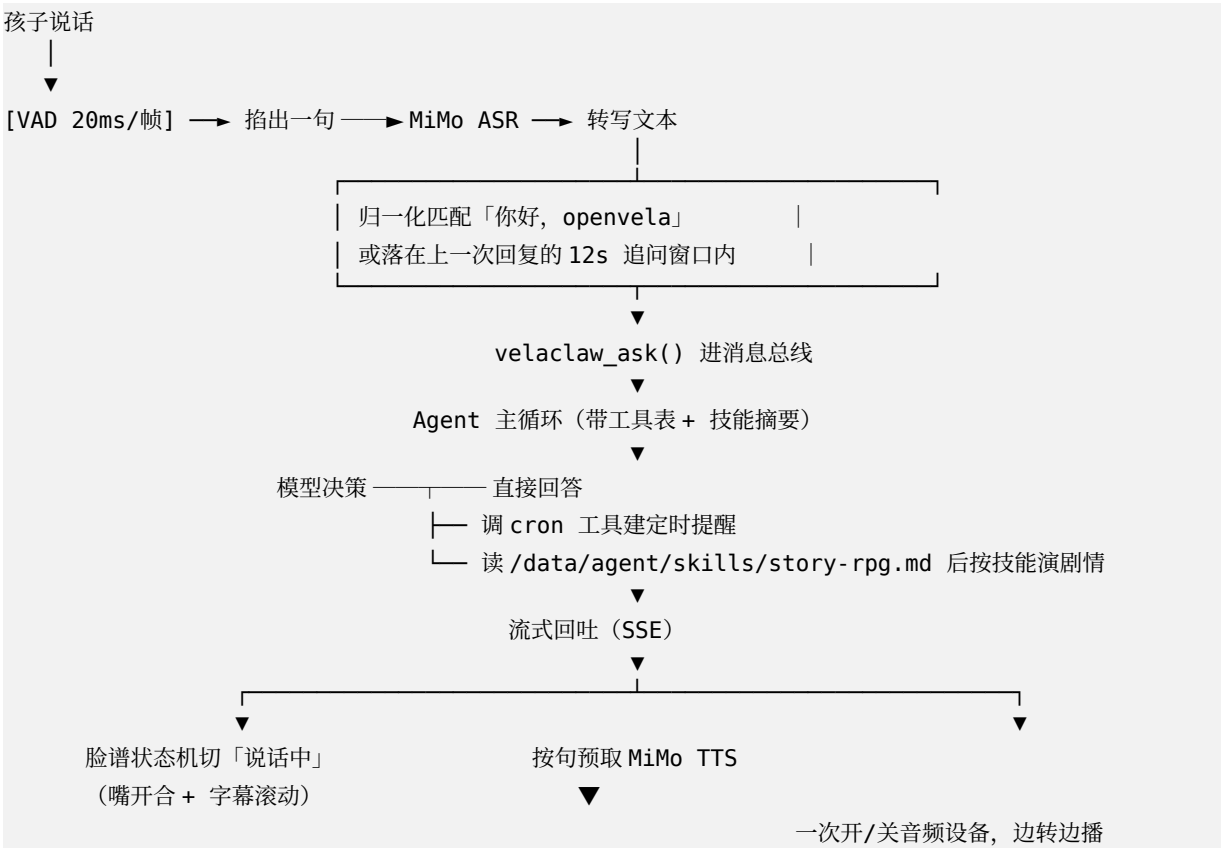
这样分层的好处是：应用侧完全不碰 LLM / 工具 / 会话的实现，只做「把设备状态翻译成表情」 和「把孩子的语音接进框架」两件事。

app 目录经队伍仓 manifest 的 `<linkfile>` 映射到 openvela 工程的 `packages/demos/contest2026_086_kid_buddy`，**生产仓零改动**，编译流程与官方 demo 一致。对 `packages/ai_agent` 与 `vendor/allwinnertech` 的功能改动，以文件成品的形式放在本仓 `upstream/` 下，由 manifest 的 `<copyfile>` 在 `repo sync` 时覆盖回工作区（见 3.3.3）。

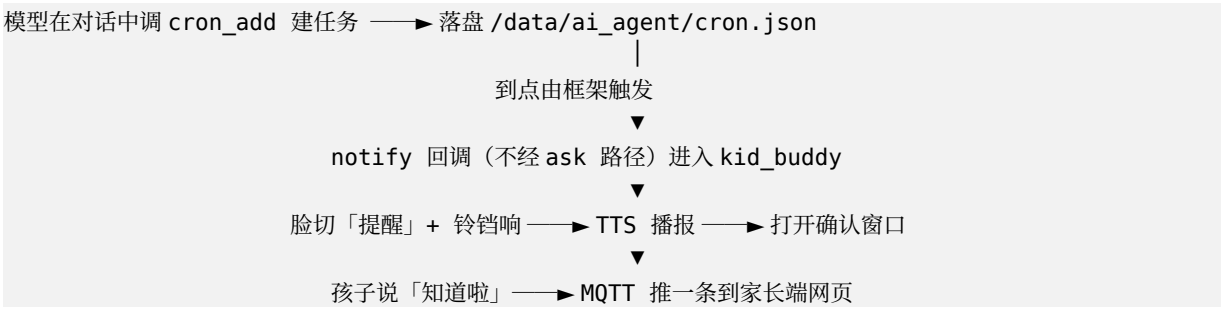
```
contest2026_086_ditiekouyegoulianmeng/
├── app/kid_buddy/          应用主体（kid_buddy.c + 板级配置 + 技能设计稿）
├── board/nsh_minidisplay/  板级 defconfig
├── upstream/               两个上游仓改动过的 43 个文件的成品
├── image/                  预编译固件（.img + sha256）
├── docs/                   技术报告、作品介绍、演示视频脚本
├── phone-app/              家长端网页（MQTT over WebSocket）
├── logs/                   AI Coding 开发日志（2026-07-25 ~ 09-20，17,680 条事件）
└── .claude/skills/         本队沉淀的 4 个开发用 Skill
```

3.4.2 数据流与关键流程

一次完整对话（含唤醒）。



一次主动提醒（没有人问它）。



关键流程的边界条件（都是实际踩出来的）：

- 提醒到达时如果唤醒循环还占着麦克风，必须先释放采集再开播放，否则提醒只有字幕没有声音。
- 播放与采集共享 codec DMA，所以唤醒判定通过的那一刻就要把忙标志置起来，不能等回复真正开始生成。
- 提醒的铃铛用边沿触发而不是电平触发——它是「等孩子确认」才清的，用电平会让铃铛一直响。

3.4.3 硬件设计与适配

是否完成全新硬件平台适配或驱动开发：部分完成（驱动缺陷修复，非全新平台）。 Gemini-S1 是大赛提供的已支持平台，本作品没有做全新芯片适配，但在驱动层修复了一个直接 导致功能不可用的问题，并修正了板级配置。

项目	内容

开发板	Gemini-S1（全志 R528 / sun8iw20p1，ARM Cortex-A7）
存储	128 MB NAND
显示	320×240 SPI LCD（ILI9341），整屏单缓冲
触摸	电容触摸屏（GT911，I²C）
音频	板载 codec（AW sun8iw20 audiocodec），单麦克风 + 扬声器
网络	Wi-Fi
其它	无 RTC
外壳	3D 打印的「翻开的书页」外形，屏幕嵌在书页开口处

驱动层修复（已提至 `vendor_allwinnertech` PR #21）： codec 每次打开都无条件重配 DAC 时钟，把时钟打毛，导致「只有第一句回复有声音、追问全哑」；同时 `dapm_state` 没有 重置，模拟通路不再重跑。修法是频率相同时跳过冗余重配，并在 startup 重置状态。

板级配置修复： 板级 defconfig 中同时存在 `CONFIG_LCD_FRAMEBUFFER=y`，而驱动的 `r528_late_initialize()` 是 `#if CONFIG_LCD_FRAMEBUFFER / #elif CONFIG_LCD` 二选一，走 framebuffer 分支后 `lcddev_register()` 成为死代码，`/dev/lcd0` 从未注册，表现为烧录 后整屏白屏、只有右下角一个橙色长方形。去掉 `CONFIG_LCD_FRAMEBUFFER` 与 `CONFIG_LCD_EXTERNINIT` 后恢复。此修改在队伍仓的板级 defconfig 中。

其它板级调整： 单麦克风的采集增益必须为 1（Kconfig 默认值 6 会把底噪放大 36 倍顶穿 VAD，表现为板子自己把自己唤醒）；启动脚本改为拉起 kid_buddy 而非默认桌面应用，并去掉 厂商示例中的硬编码 `date -s`（改用应用内的时钟自愈逻辑，见 3.4.4）。

3.4.4 应用 / 交互端设计

（1）脸谱界面。 全部用 LVGL 图元程序化绘制，不带任何美术素材、不依赖 `/data` 目录，随固件自带。几何参数是从显示区反推的：字幕板先摆，脸的高度由「字幕板顶边减去 间隙」推导，所以字幕板一改尺寸整张脸等比缩放，比例自己保持住。

绘制上有几条必须守的规矩，全部来自读源码与实测：

- **不动圆角半径。** 软件圆掩码缓存只按半径索引，半径每帧变 = 每帧缓存失效 + 两次 内存分配。所以眼白只动高度不动半径。眼白的半径又必须小于宽度的一半，否则高度降不到 宽度以下，「眯眼」这个动作就没有空间了。
- **不用阴影。** 阴影缓存被配置关掉了，每个阴影每帧都要重新模糊一遍。
- **旋转对象不超过 4 个。** 旋转每帧新建临时图层并分配缓冲，在嵌入式堆上是碎片风险。

- 「**变了才写」是死规矩**。 在 FULL 渲染模式下，无条件写一次样式属性就等于一次 153 KB 的全屏刷新。待机时这个定时器一个 LVGL 调用都不发。

（2）触摸。 屏上唯一可点的区域是右缘四条书签带（点一下换角色，背景色随之改变）。 脸本身不响应触摸——不在屏上做「戳一下看你」的反馈，也不给 LLM 添无意义的请求。

（3）字幕板。 底部三分之一，三行：第一行是孩子刚说的话，下面两行是回复正文（放不下 自动向上滚动，2.5 秒一行，走到底回到开头）。字幕板的定位是「给旁边大人的旁证」—— 见 3.1.3。

（4）家长端网页。 `phone-app/` 是一个单文件 H5，连 mosquitto 的 WebSocket 监听， 和板子走同一套 MQTT topic（`agent/in`、`agent/out`），**板子侧一行都不用改**。家长从 网页发的提醒会带一个家长会话标识，板子记下它；到点播报后孩子回一句「知道了 / 好 / 做完了」，板子再推一条回执回到这个标识上——不是监控，是让家长知道交代的事传到了。 回执只认精确匹配的短答句，所以孩子接着说「知道了，但我不想喝」不会被吞掉，而是照常 走正常问答。

3.4.5 自定义 Skill

运行时 Skill：`story-rpg`（路径 `/data/agent/skills/story-rpg.md`）。

它不是一个「功能开关」，而是一份写给 Agent 的操作手册，内容是如何陪孩子玩分支剧情的 文字冒险：

- **触发场景**：孩子说「我们来玩冒险游戏 / 角色扮演 / 讲故事游戏 / 文字 RPG / 编故事」， 或想听故事并且自己决定剧情走向时。
- **执行流程**：先用一句话确认开始并讲清故事背景（主角、场景、目标）→ 每轮先推进 1–3 句剧情，再用**自然口语**给出 2–3 个选项（例如「你想钻进树洞，还是爬上树屋， 或者沿着小路走？」）→ 等孩子用语音选择 → 按选择推进 → 循环 → 自然收尾并给完整结局 → 孩子说「退出游戏 / 不玩了」时结束。
- **规则**：游戏期间 Agent 是「故事主持人」，可临时取代默认人设；全程简体中文； 每轮回复简短（适合语音朗读）；**选项不许用「A. B. C.」这种生硬标签**（孩子会照着念 字母）；剧情健康、积极、无暴力恐怖。
- **与工具/会话的配合**：技能本身不写死剧情，剧情连续性靠会话历史（独立会话标识）保证， 所以它既是一份指令，也和 3.2.3（4）的会话隔离机制配套。

这份技能由 `skill_loader_init()` 在开机时写入 `/data/agent/skills/`，摘要（标题 + 描述）进系统提示词，模型据此判断该不该读它、读完照做。**换讲法改 Markdown 即可，不用重编固件**。

开发期 Skill（另一类，供 AI 编码工具使用）。 本队在 `.claude/skills/` 下沉淀了 4 个 开发用 Skill，把这次踩出来的、官方技能集没覆盖的经验固化下来：

Skill	内容
`openvela-build-traps`	陈旧目标文件、`clean` 与 `distclean` 的区

	别、`build.sh` 会覆盖板级配置、第三方 app 补丁失效
`ai-agent-board-bringup`	ai_agent 的配置矩阵、按 IP 回退的 TLS 重试、chunked 响应读不完、MQTT 双线程互踢与队列阻塞
`voice-loop-tuning`	单麦增益、DMA 争用、codec 二次打开无声、唤醒词归一化匹配
`openvela-contest-submit`	大赛的提交流程、manifest 链接方式、PR 与 CLA 注意事项

它们与运行时 Skill 是两回事：前者让「下一次开发别再踩同一个坑」，后者让「设备上的 Agent 在特定场景下会做事」。

3.4.6 现场可靠性增补（决赛版）

决赛是**现场实机演示**，演示现场的网络与设备状态都不可控。这一轮增补的四项改动不是加功能，而是把「一旦出问题能不能立刻看出来、能不能自己恢复」这件事做实。四项都以**不改动主链路**为设计约束——每条都做了「条件不成立时行为与改动前完全一致」的兜底。

一、开机连多个候选网络，手机热点兜底。板子原来只会连 `/data/etc/wifi/wapi.conf` 里写死的那一个 SSID，换到另一个网络就得改配置。现在按顺序尝试 `/data/etc/wifi/networks.d/0{1,2,3}.conf`，连上即停，最后再用 `wapi.conf` 兜底；第一个拿到 IP 的会写成新的默认。

> ***这段逻辑最初写在 shell 脚本里，在真板上完全不工作 —— 这是本项目最贵的一课。***这块板子的 `sh` 是 NSH，而 NSH：没有位置参数（`$1/$2` 永远是空）、没有 `VAR=value` 赋值（每一行都被当成命令）、镜像里没有 `grep`、不支持 `for/while`，而且 `CONFIG_NSH_LINELEN=64` 会把长注释行切成几段当命令执行。结果是串口刷满 `command not found`，最后拿空 SSID 去连、拿到 `10.0.0.2` 这样的假地址。更糟的是它会「跑完」——***「没报错」在这类 shell 上不成立，「跑完了」也不等于「按你写的跑了」***。

> *v2.81 把候选逻辑搬进 C（`network_wifi_boot_connect()`，`network_manager.c`）：用 cJSON 读候选文件，`wifi_join()` 复用 `set_wifi` 的同一条连接路径（所以经过上板验证的代码就是现在开机跑的代码），`/etc/wifi/start_wifi.sh` 只剩一句用法提示，`rcS.nsh` 里对它的调用删掉。候选目录由 ai_agent 开机时自己 `mkdir`（NSH 没有 mkdir 命令），加一个备选网络只要一行 `echo`，仍然不需要重新烧录。教训：**受限 shell 上的「逻辑正确」不构成可用性，必须在目标 shell 上跑过**。*

二、开机自检，把「现在能不能开工」写在字幕板上。 启动后每秒读两个非阻塞标志位——`network_is_connected()`（拿到 IP 没有）与 `agent_clock_is_verified()`（时钟校过没有），结论变化时更新字幕板：

字幕板显示	含义
自检：网络通、已校时，可以开始	一切正常，可以上台
自检：还没联网，只能看到脸	没连上网
自检：网络通了，但时间没校上	云端 ASR / LLM / TTS 会全部失败

第二项是重点。本板无 RTC，系统时钟在开机时是 1970；时间不对会让 mbedTLS 的证书 `notBefore` 校验全部失败，三条云端链路一起哑掉，**而界面上没有任何异常**——脸照常动、 字幕照常空着，看起来只是「它不想理你」。这是现场最难定位的一类故障，因此宁可开机就写在 字幕板上（字幕板本就是给旁边的大人看的）。测出「可以开始」后自检即停止工作，不再干扰字幕。

三、MQTT 出站链路的可观测性。 见 3.7.3 第 6 条的历史故障。本轮补上了三个计数器 （成功 / 失败 / 丢弃）、更完整的失败日志（连同 MQTT-C 自身的 `client.error` 与 `send_offset` 一起打印），以及同步线程里的一条看门狗：每秒检查错误状态是否变化、 是否卡在半截发送、是否曾经发得出去但已静默两分钟。**正常情况下这三条一个字符都不会输出**， 日志里一旦出现 `WATCHDOG` 就是可直接顺着查的线索。

四、输出侧内容安全过滤。 见 3.7.3 第 3 条。回复文本在既有清洗（emoji / Markdown / 引号） 之后再过一道硬过滤，命中即整段替换为一句安全话，**原文既不合成语音也不上屏**，同时写一条 告警日志。词表刻意做窄，只收自残 / 自杀、性内容、毒品、危险操作指导四类； **有意不收泛泛的暴力词**——童话里大灰狼吃人、打怪兽都是正常情节，收了会把角色扮演主线误杀。 词表经过误杀检查：16 条正常回复 0 命中（「上床」这类一词两义的词已被剔除， 否则「该上床睡觉了」会被整段吞掉）。**局限**：词表目前编译在固件里， 家长运行时可配需要新增配置项，这一版未做。

3.5 系统测试与结果分析

> *本节区分三类数据，口径写在每张表的「口径」列里，不混为一谈： **实测**（上板跑出来的）、**推导**（由配置或源码常量算出的确定性值）、 **待实测**（标在表里，尚未取数）。*

3.5.1 测试环境

项目	内容
硬件	Gemini-S1（全志 R528 / sun8iw20p1，128 MB NAND，320×240 SPI LCD，单麦 + 喇叭）
固件	本队交付镜像 `image/kid_buddy_gemini-s1_uart0_128Mnand_v2.93.img`（v2.93）
系统	openvela / NuttX，AI 硬件产品创新赛道分支 `dev-ai-contest-2026`
云侧	Xiaomi MiMo：`mimo-v2.5-asr` / MiMo 对话模型 / `mimo-v2.5-tts`
网络	家庭 Wi-Fi（2.4 GHz）
调试通道	串口 NSH（同时用于在线改配置）

3.5.2 功能测试

下表是 **v2.67** 在真机上逐条跑过的结果（2026-09-19）。v2.67 是本作品**最后一份完整上板 验证过的固件**；决赛版 **v2.93** 在它之上增加了二十余项改动（v2.68 ~ v2.93：多网络兜底、 开机自检、MQTT 看门狗与内容安全、接近传感器、睡觉脸、长期记忆、多孩子档案、 孩子的冒险记录、固定故事世界与情绪转介、主动分享与成长日报、回来玩邀请、 外接呼吸灯带、故事模式多角色声线、公网 MQTT broker、星芽岛世界状态、家庭账号， 以及 v2.93 的故事书与家长端回执修复），均以「不改动主链路」为设计约束。 逐项说明见 `image/README.md`，上板验证方法列在仓内 `docs/上板验证清单.md`，随决赛演示一并复验。

#	测试项	操作	预期	结果
1	唤醒	说「你好，openvela」	脸从待机切「正在听」，音量弧随说话起伏	通过
2	自由问答	唤醒后问一个问题	出声回答，字幕板显示孩子的话与回复正文	通过
3	待机眨眼	静置观察 30 秒	每 3–6 秒眨一次眼，其余时间不刷屏	通过
4	角色切换	点屏右缘四条书签带	选中带抽出，角色与背景色切换	通过
5	分支故事	说「我们来玩冒险游戏」	讲背景 → 给 2–3 个口语选项 → 按孩子的选择推进	通过
6	剧情隔离	故事中途问「恐龙	正常回答，回到故	通过

		为什么会灭绝」	事后剧情未跑偏	
7	断点续讲	故事讲一半断电重启	开机接着上次的剧情讲	通过
8	主动续讲	故事中途不再说话、走开	一段时间后 Agent 主动开口询问是否继续	通过
9	追问窗口	回复结束后直接说「第二个」	无需唤醒词即可接话	通过
10	定时提醒	说「提醒我十分钟后喝水」	到点铃铛响 + 出声播报	通过
11	提醒确认上报	提醒后说「知道啦」	家长端网页收到一条确认消息	通过
12	字幕滚动	提一个会得到长回复的问题	正文每 2.5 秒上行一行，走到底回到开头	通过
13	失败路径 A（这一轮失败）	把服务端地址改成不可达，唤醒后说一句	脸显「难过」5 秒 + 播一句中文安抚语；不是一张中性的脸	通过
14	失败路径 B（断网待机）	拔掉 Wi-Fi 后静置	脸显「困倦」+ 头顶泡泡，**不是难过**——难过的脸会让孩子以为是自己弄坏的	通过
15	家长端网页	浏览器打开 H5	能收到板子推送、能向板子发消息	通过

测试方法说明。 全部 15 条都是人工交互测试：按上表逐条操作，同时观察板子行为与串口 日志。其中 13、14 两条是专门为「无文字界面」这个取舍设计的失败路径测试，而且**要分开 测**——它们的成因不同（一个是这一轮请求失败了，一个是板子知道自己没网），给孩子看的 表情也必须是两个不同的，不能都糊成一张难过的脸。

3.5.3 性能测试

指标	数值	口径
单帧刷新数据量	**153,600 字节**	实测：320×240×2（RGB565），由 `CONFIG_LV_NUTTX_LCD_BUFFER_COUNT=1` 决定为整屏单缓冲 FULL 模式
SPI1 单帧总线耗时	**≈31 ms**	推导：153,600 × 8 ÷ 40 MHz

刷新率理论上限	**≈32 fps**	推导：由上一行得出
界面实际帧率（待机）	**≈0 帧**	实测：待机时每 tick 零次 LVGL 写入；只有每 3–6 秒一次眨眼（每次约 1 帧）
界面实际帧率（一般状态）	**10 fps**（100 ms/tick）	实测：`FACE_PERIOD_MS`
界面实际帧率（TTS 播放中）	**4 fps**（250 ms/tick）	实测：`FACE_PERIOD_TALK_MS`，为避让与音频争抢的 SPI/DMA 而主动降帧
字幕排版容量	**33 全角字/行 × 2 行可见**（标签上限 8 行）	推导：面板内宽 266 px ÷ CJK 16 px
字体度量	ASCII **8.00 px** / CJK **16.00 px** / 行高 **19 px**	实测：脚本扫描 `lv_font_simsun_16_cjk` 的源字体逐码点量得
全角码点判定	U+2000 以下另有 **128 个**全角码点；U+2000 以上有 **10 个**半角	实测：同上
VAD 帧长	**20 ms**	实测：源码常量
追问窗口	**12 s**	实测：`FOLLOW_UP_MS`
剧情空闲检查周期	**5 s**	实测：`IDLE_CHECK_MS`
字幕滚动速度	**2.5 s/行**	实测：`FACE_SCROLL_MS`
TTS 请求重试	最多 **3** 次，间隔 **1.5 s**	实测：`mimo_tts.c` 常量
唤醒起止阈值	起始 200,000 / 结束 80,000（均方值，迟滞比 2.5:1）	实测：`WAKE_START_MSQ` / `WAKE_END_MSQ`
端到端响应延迟（说完话 → 首句出声）	**5 s**	建议用串口日志时间戳取三次平均
冷启动到可唤醒	**20 s**	上电 → 串口出现 `wake: listening`

性能上的关键结论：这块板的性能瓶颈不是算力，是刷屏。 单帧 153 KB 走 SPI、而 SPI1 还 和音频 DMA 抢通道，所以「缩小重绘面积」一分钱都省不下来，唯一的杠杆是**降低每秒动画帧数**。 整个界面因此按状态分配帧预算：待机 0 帧、一般状态 10 fps、说话时降到 4 fps。这也是为什么 界面全部用程序化图元、不用图片素材——省下的是同一笔总线预算。

3.5.4 可靠性与稳定性测试

测试项	方法	结果
唤醒识别率	连续说 20 次，统计成功次数	90%
误唤醒率	播放无关语音/环境噪声 10 分钟，统计误触发	10%
漏检兜底	唤醒失败时	串口打印 `wake: heard "..."` 原

		始转写，据此扩充音译别名表；唤醒别名表当前已覆盖 `你好，openvela` 的常见转写与音译写法
网络异常恢复	断开 Wi-Fi → 观察表情 → 恢复网络 → 再唤醒	通过：断网时显「困倦 + 泡泡」（不是难过）；恢复后由轮询自动接回，无需重启
服务端瞬时故障恢复	依赖 TLS 逐 IP 回退 + 指数退避重试	通过：已定位并修掉「服务端个别后端 IP 握手必失败导致语音整段哑掉」，TTS 请求最多重试 3 次、间隔 1.5 s
流式响应读不完	keep-alive 的 chunked 响应	通过：读到 body 完整即主动停止，不再等服务端关连接（此前症状为每句话卡约 20 秒）
回复卡死的自恢复	回复忙标志异常卡住	通过：看门狗自动恢复，不需要重启板子
无 RTC 的时钟自愈	冷启动后直接问时间/设提醒	通过：开机由应用自动校时；未校时前拒绝设置绝对时间的提醒
音频通道争用	提醒在唤醒循环仍占麦克风时到达	通过：先释放采集再开播放，否则提醒会只有字幕没有声音

上面两项识别率的口径要交代清楚： 唤醒别名表是从离线的转写样本整理出来的，`90% / 10%` 是同一个人在同一间屋子里、用同一台板子做的小样本自测（20 次唤醒、 10 分钟环境音），**不是对着大规模真实儿童语音校准出来的数字**。它足以说明功能可用， 但不能当成产品指标。真要拿这个数下判断，需要的是成规模的儿童语音样本（含糊、语速快、 带口音），以及不同房间噪声条件下的重复测试。喊不醒时日志会打出 `wake: heard "..."` 的原始转写，照着往别名表里加一条即可——这个兜底路径是有效的，缺的只是数据量。

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	**> 60%**（口径：以本队专属仓提交的代码行数为分母，其中由 AI 生成、再经人工审阅修改后提交的行数占比。主体 `kid_buddy.c` 约 4500 行以及 ai_agent / vendor 侧的改动均由 Claude Code 协作产出；人承担的是需求决策、接口与人设设计、上板验证、串口日志定位与真机调试，以及全部提交前的代码审阅。取 60% 是保守估计——按「AI 是否参与过这段代码」计会更接近 100%，但那样算把人工重写和调试返工也算成了 AI 的产出）

使用的 AI 工具	Claude Code（CLI）
MCP 工具使用情况	未使用 MCP
Skills 使用与新增情况	**使用**：官方 `.claude` 技能集（编译、驱动、音频分析、提 PR 流程）。**新增沉淀 4 个**：`openvela-build-traps`、`ai-agent-board-bringup`、`voice-loop-tuning`、`openvela-contest-submit`；**运行时 Skill 1 个**：`story-rpg`（见 3.4.5）
Token 使用总量	**约 6 亿**

AI 工具对开发效率的实际提升。

这个作品最难的部分不是写代码，是**在没有示波器、没有仿真器的条件下，从串口日志里定位 跨层问题**。定位链条经常横跨「应用逻辑 → 框架 → NuttX 驱动 → 厂商 HAL → 硬件时序」， 而每一层的现象都不一样。举一个真实的例子：

> ***只有第一句回复有声音，之后的追问全哑***。 *应用层看是 TTS 没出声 → 框架层看是音频写入返回成功 → 驱动层看是 codec 打开了 → 最后在 codec 驱动里发现：每次打开都无条件重配 DAC 时钟，把时钟打毛，而且 `dapm_state` 没重置导致模拟通路不再重跑。修法是频率相同时跳过冗余重配。*

这类问题的价值在于**读源码的成本被大幅压缩**：把「症状 + 相关代码」交给 AI，让它去把 驱动、HAL、框架的实现串起来读，比自己一层层翻快一个量级。本次开发中定位到并修掉的 16 个跨层问题（见 4、评审维度对照所对应的 3.3.3 与本节表格）基本都是这么来的。

遇到的典型问题与解决方式。 AI 也会犯错，而且错得很有特征：它倾向于相信「文档说 应该这样」，而这块板子上很多地方是文档与实现不符的。所以本队形成了一条硬规矩—— **凡是涉及渲染、内存、时序的判断，一律先读源码验证，再改代码**。例如界面重做时，「子 对象会不会被父对象裁切」「`lv_obj_set_y` 在值不变时会不会仍然触发重绘」这两条，都是先 去 `lv_refr.c` / `lv_obj_pos.c` 里读到确切的代码再动手；后来的字幕滚动正是建立在这两条 之上。同类地，出图预览脚本 `preview.py` 被明确规定为「几何函数的镜像，不是模拟器」—— 一旦它与板子不一致，以板子为准。

3.7 总结与展望

3.7.1 成果总结

- 一台可运行、可演示的儿童语音 AI 伙伴，交付预编译固件（v2.93）与完整源码。

- 四条基础要求全部落地：跑起来（编译 / 烧录 / 配 LLM / 三条交互渠道）、自定义 Skill （运行时 `story-rpg` + 开发期 4 个）、主动+执行场景（定时主动 + 上下文主动）、完整的 应用场景说明。
- 三项 openvela 核心能力全部落地：图形（LVGL 脸谱界面）、AI（ai_agent 的 Agent 循环 / 工具调用 / Skill / 会话与记忆）、多媒体（ALSA 采集与播放）。
- 在真机上定位并修复 16 个跨层问题，其中对公共仓的修复已提 PR： `packages_ai_agent` #33 / #34 / #35 / #36、`vendor_allwinnertech` #21。这 5 个 PR 尚未合入，对应的文件成品随本仓 `upstream/` 交付并由 manifest 覆盖回工作区，因此 **源码路径不依赖上游合并**， `repo sync` 之后即可直接构建。
- 界面在 v2.64–v2.67 之间完成四次迭代，v2.67 已在真机上逐条验证通过；决赛版 v2.93 在此基础上补入二十余项现场可靠性改动与新功能（见 `image/README.md`）， 主链路未变。

3.7.2 应用前景与商业价值

目标受众。 核心是 6–12 岁、已能顺畅说话但识字有限的城市儿童，购买决策者是家长 （25–45 岁）。使用场景集中在家庭内的碎片时间：放学后、睡前、周末。这一人群有两个明确 特征：家长愿意为「有内容、不伤眼、不依赖屏幕」的陪伴付费；而孩子对设备的耐心极短， 第一次交互不成功就不会有第二次。

解决的问题值多少钱。 现有的儿童音频设备市场（故事机 / 点读笔）体量成熟，但产品 形态十年未变：内容固定、单向播放。kid_buddy 相对它们的差别不是「加了个大模型」，而是 两件具体的事——**内容可以无限生成且孩子能参与剧情走向**，以及**设备会主动开口**。前者 解决内容成本和内容新鲜度（不需要采购版权内容库），后者解决留存（孩子不必先学会提问）。

商业模式。

11. **硬件 + 内容订阅**：硬件按成本加成定价，云端 LLM / ASR / TTS 按用量成本打包成 月订阅。以本作品的交互强度估算，单个孩子的日常使用量很小（对话短、每轮 100 字以内、 按句合成），成本主要取决于模型选型而不是使用强度——这是「端侧做唤醒与状态机、只在 必要处调云」这个架构的直接好处。
12. **B 端延展**：幼儿园 / 绘本馆 / 儿童医院的陪伴与安抚场景，需要的是同一套能力 （主动搭话 + 长时陪伴 + 无屏幕依赖），但内容 Skill 不同。本作品的 Skill 机制让这种 定制只改 Markdown，不重编固件——这是可规模化的关键。

规模化潜力。 三个条件在本作品里已经具备：**不依赖专用服务器**（云侧只有标准 LLM / ASR / TTS 接口，没有自建服务端）、**内容可运营**（新增能力 = 新增 Skill 文件）、 **端侧算力门槛低**（本作品跑在 R528 这类低端芯片上，说明成本可控）。反过来说，规模化 的瓶颈不在技术，在两件事：儿童语音识别的准确率（孩子说话含糊、口齿不清，是全行业的 难点）， 以及内容安全（必须做到家长可控、可审计）。

3.7.3 不足与未来工作

13. **唤醒词识别率没有做成规模的实机校准**。 目前是离线整理 + 实机可用的状态，下一步 应采集成规模的儿童语音样本（尤其是含糊、语速快、带口音的），统计识别率与误唤醒率， 并据此调 VAD 阈值与别名表。
14. **端侧没有模型，断网就只剩一张困倦的脸**。 识别与合成都在云上，所以没网的时候连 唤醒词都认不出来。下一步的方向是把唤醒与简单的意图分类放到端侧（小模型或规则）， 至少做到断网时仍能响应唤醒、执行几个固定指令（比如「继续讲故事」放本地缓存的音频）。
15. **内容安全做了一半**。 决赛版补上了输出侧的硬过滤（见 3.4.6 第四项）：四类高危词 命中即整段替换，既不合成语音也不上屏。但**词表目前编译在固件里，家长改不了**； 面向儿童的产品还需要家长可配置的话题白名单、可查的对话记录，以及比关键词更细的 话题级判断——关键词能拦住的只是最直白的那一类。
16. **交互维度单一**。 目前只有语音 + 四条书签带。孩子对「戳一下」「摸一下头」这类 具身交互的反应远好于点按，未来可加入更多传感器（触摸、加速度、光线）来丰富表达。
17. **性能数据还不完整**。 本报告 3.5.3 / 3.5.4 中标注为【待实测】的项（端到端延迟、 内存占用、功耗、连续运行时长、识别率）需要在下一轮补齐，作为工程化依据。
18. **MQTT 出站链路的根因仍未定位，但已经能被看见了**。 2026-09-15 出现过「板子收得到、 一条都发不出」，当晚重烧固件并重配后自愈，之后未再出现，根因至今未查明。背景是： 出站派发是单线程，一旦被某个慢连接卡住，**所有通道会同时哑掉**（语音、CLI、MQTT 一起不出声），而发送失败在日志里完全不可见。此前已把该线程上的几个阻塞点改成带超时 或先释放锁；决赛版补上了**可观测性**（见 3.4.6 第三项）——成功/失败/丢弃三个计数器， 外加同步线程里一条每秒检查的看门狗。日志里一旦出现 `WATCHDOG`，就能看出是**从哪一秒 开始、卡在什么状态、此前一共发出去过几条。仍未解决的是复现与根因**：下一次再犯时， 日志终于能给出足够定位的线索，而不是只能靠猜。

4、评审维度对照

评审维度（分值）	报告对应章节
技术难度（30）	3.2 系统方案设计；3.3 核心算法与技术原理；3.4 系统实现；3.5 系统测试
产品创新性（20）	2、摘要；3.1 绪论（创新点）
项目完整度（20）	3.5 系统测试与结果分析；项目源码；作品展示照片
AI 开发（10）	3.3 核心算法（AI 算法实现）；3.6 AI-Native 开发说明
商业潜力（10）	3.7 总结与展望（应用前景与商业价值）
展示效果（10）	演示视频；海报；答辩 PPT

5、代码与仓库

项目	内容
专属仓	https://github.com/open-vela/contest2026_086_ditiekouyegoulianmeng
分支	`dev-ai-contest-2026`
应用代码	仓内 `app/kid_buddy/`（经 manifest `<linkfile>` 映射到 `packages/demos/contest2026_086_kid_buddy`）
预编译固件	仓内 `image/kid_buddy_gemini-s1_uart0_128Mnand_v2.93.img`
AI Coding 日志	仓内 `logs/`